

# CURSO DE PYTHON

**Corporación Ecuatoriana para el Desarrollo de la  
Investigación y la Academia**

**Ing. María Fernanda Granda J., PhD**

**22-07-2026**

**“Conectando ideas,  
transformando  
sociedades”**

## **Tema 5: Calidad de Código, Manejo de Errores, Debugging y Testing con Pytest**

### **Objetivo de Aprendizaje:**

Al finalizar la sesión, el estudiante será capaz de desarrollar aplicaciones más robustas mediante el manejo adecuado de excepciones, técnicas de depuración, registro de eventos (logging) y pruebas automatizadas con Pytest, aplicando principios de calidad de software utilizados en entornos profesionales.

### **Competencias:**

- Identificar errores comunes en aplicaciones Python.
- Implementar manejo de excepciones para evitar fallos inesperados.
- Utilizar herramientas de depuración para encontrar errores.
- Registrar eventos y errores mediante logging.
- Crear pruebas unitarias automatizadas con Pytest.

- Aplicar buenas prácticas de calidad de software.

## **Tema 5: Calidad de Código, Manejo de Errores, Debugging y Testing con Pytest**

- Manejo de excepciones
- Debugging
- Logging
- Pytest
- Los 5 Principios SOLID

## ¿Por qué la calidad del software es importante?

- Reduce drásticamente los costos a largo plazo, minimiza los fallos operativos, y garantiza la seguridad de los datos.
- Genera fidelidad del cliente y protege la reputación de la empresa.

Se destaca por aportar:

- **Optimización de Costos:** Detectar y corregir errores tempranamente, esto optimiza los recursos de mantenimiento.
- **Seguridad y Fiabilidad:** Previene vulnerabilidades, pérdidas de información y caídas del sistema, asegurando operaciones continuas sin interrupciones.
- **Experiencia de Usuario (UX):** Un sistema intuitivo, rápido y libre de errores incrementa la adopción y la satisfacción del cliente final.
- **Escalabilidad y Mantenibilidad:** Un código estructurado bajo buenas prácticas facilita la integración de nuevas funciones y la adaptación a las necesidades cambiantes del mercado.

## ¿Cómo lograr mejorar la calidad del software?

Lograr un software robusto en Python requiere:

- Aplicar buenas prácticas de diseño,
- Manejar los fallos de manera controlada y
- Validar el comportamiento del sistema mediante pruebas automatizadas.

Las estrategias clave para optimizar la calidad del código, el debugging y el testing:

- La calidad del código se basa en la legibilidad, la mantenibilidad y el cumplimiento de estándares.

## ¿Cómo lograr mejorar la calidad del software?

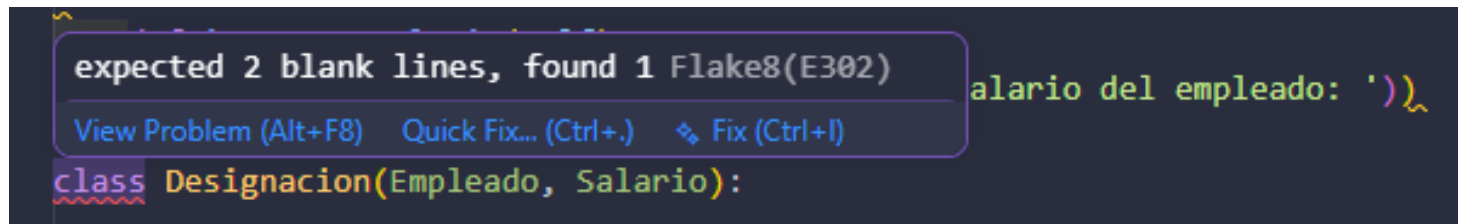
Para garantizar que el código siga las convenciones (PEP 8) y evitar errores comunes, se puede usar herramientas automatizadas:

- **Linters (Flake8 / Ruff):** Analizan el código estáticamente en busca de variables no utilizadas, sintaxis incorrecta y problemas de complejidad ciclomática.
- **Formateadores (Black / Ruff):** Aplican un formato consistente y sin debates sobre espacios o indentación, ajustando el código automáticamente al estándar oficial.
- **Ordenadores de imports (isort):** Organizan y agrupan automáticamente las librerías estándar, de terceros y locales en bloques limpios.

## Uso de flake

- **Instalación de flake8:** en la terminal integrada de VSC pip install flake8.
- **Instalar la extensión oficial:** En la pestaña de Extensiones en la barra lateral de tu VS Code (o presiona Ctrl + Shift + X), busca Flake8 (publicada por Microsoft) y haz clic en Instalar
- **Habilitar como Linter:** Abrir la paleta de comandos presionando Ctrl + Shift + P (o Cmd + Shift + P en Mac). Escribir y seleccionar Python: Select Linter. Elegir flake8 de la lista.
- A partir de este momento, VS Code evaluará tu código automáticamente al editar o guardar el archivo.

Ejemplo:



The screenshot shows a VS Code editor with a dark theme. A tooltip for a Flake8 error is displayed, stating 'expected 2 blank lines, found 1 Flake8(E302)'. Below the message are three links: 'View Problem (Alt+F8)', 'Quick Fix... (Ctrl+.)', and 'Fix (Ctrl+I)'. In the background, a Python class definition is visible: `class Designacion(Empleado, Salario):`. To the right of the error message, a snippet of code is visible: `salario del empleado: '))`.

## Uso de black

### Instalar black:

- En la terminal integrada de VSC pip install black.

### Corregir los problemas de estilo:

- black archivo.py



## Uso de isort

- **Instalación:** en la consola de VSC pip install isort.
- **Uso:** en la consola usar el comando isort tu\_archivo.py. Si prefieres ordenar todos los archivos de tu proyecto de una sola vez, ejecuta: isort .

Ejemplo:

### Antes de isort

```
import os
from django.conf import settings
import sys
```

### Después de usar isort

```
import os
import sys

from django.conf import settings
```

## Manejo de Errores

- Un buen manejo de errores previene caídas abruptas y hace que la aplicación sea predecible.
- Las estrategias fundamentales incluyen:
- **Bloques try-except estructurados:** Captura solo excepciones específicas (ej. ValueError, KeyError) en lugar de usar un except Exception: genérico que podría ocultar fallos críticos.
- **Excepciones personalizadas:** Crea clases de error propias heredadas de Exception para dar contexto específico a tu negocio o dominio.
- **Uso de raise:** Eleva excepciones deliberadamente en capas inferiores para que sean manejadas o registradas en la interfaz de usuario o el controlador principal.

## Contexto del problema

Revisar el siguiente código:

```
edad = int(input("Ingrese su edad: "))  
  
print("El próximo año tendrá", edad + 1)
```

- ¿Qué pasa cuando se ingresa este valor en edad?

veinte

Resultado:

```
ValueError:  
invalid literal for int()
```

Preguntas para discusión:

- ¿Por qué se produjo el error?
- ¿Qué ocurre si el programa está siendo utilizado por cientos de usuarios?

## Conceptos Clave

### **Error**

Problema detectado durante la ejecución.

### **Excepción**

Evento que interrumpe el flujo normal del programa.

### **Bug**

Defecto o error en el software.

### **Testing**

Proceso para verificar que el software funciona correctamente.

## Manejo de Excepciones

- Error sin control

```
numero = int(input("Ingrese un número: "))  
  
print(100 / numero)
```

- Posibles errores:

```
ValueError  
ZeroDivisionError
```

## Estructura Try-except

- Uso de una excepción general

```
try:  
    numero = int(input("Ingrese un número"))  
    print(100 / numero)  
  
except:  
    print("Ocurrió un error")
```

- Captura específica:

```
try:  
    numero = int(input())  
  
except ValueError:  
    print("Debe ingresar números")
```

## Múltiples excepciones

- Capturar múltiples excepciones con el mismo bloque (en una tupla)

```
try:
    valor = int(input("Introduce un número: "))
    resultado = 10 / valor
except (ValueError, ZeroDivisionError) as e:
    print(f"Ha ocurrido un error (ValueError o ZeroDivisionError): {e}")
```

- Capturar múltiples excepciones de forma separada

```
try:
    archivo = open("datos.txt", "r")
    linea = archivo.readline()
    numero = int(linea)
except FileNotFoundError:
    print("Error: El archivo no existe.")
except ValueError:
    print("Error: La primera línea del archivo no es un número válido.")
except Exception as e:
    print(f"Error inesperado: {e}")
```

## Uso de Else

- Dentro de una estructura try-except, se ejecuta solo si el bloque try no genera ninguna excepción.
- Es ideal para separar el código que puede fallar de la lógica que depende de que la operación inicial haya sido exitosa

```
try:
    resultado = 10 / 2
except ZeroDivisionError:
    print("No se puede dividir entre cero.")
else:
    # Se ejecuta solo si la división tuvo éxito
    print(f"La operación fue exitosa. Resultado: {resultado}")
```



## Uso de Finally

- El bloque **finally** en Python se ejecuta siempre al final de una estructura try-except, independientemente de si ocurrió una excepción o no.
- Se utiliza principalmente para tareas de limpieza indispensables, como cerrar archivos o conexiones a bases de datos, garantizando que estos recursos se liberen siempre.

```
try:
    archivo = open("datos.txt", "r")
    contenido = archivo.read()
    print(contenido)
except FileNotFoundError:
    print("El archivo no existe.")
finally:
    # Se ejecuta siempre, haya habido error o no
    if 'archivo' in locals() and not archivo.closed:
        archivo.close()
    print("El archivo se cerró correctamente.")
```

#Locals() devuelve un diccionario con todas las variables locales que existen en ese momento

## Lanzar excepciones personalizadas

- La palabra clave **raise** se utiliza para lanzar una excepción de forma intencional.
- Es ideal para interrumpir la ejecución ante un error y forzar su manejo.
- Su uso principal es validar datos y lanzar errores personalizados (como ValueError o TypeError) junto con un mensaje claro.
- Ejemplos:

```
edad = 15

if edad < 18:
    raise ValueError("Debe ser mayor de edad")
```

```
def dividir(a, b):
    if b == 0:
        raise ZeroDivisionError("No se puede dividir un número entre cero.")
    return a / b
```

## Lanzar excepciones personalizadas

- Para asegurar la continuidad del programa, esta excepción puede ser capturada por un bloque try...except en el nivel superior

```
try:  
    resultado = dividir(10, 0)  
except ZeroDivisionError as error:  
    print(f"¡Ocurrió un error!: {error}")
```

## Crear excepciones personalizadas

- Puedes crear tus propias excepciones personalizadas heredando de la clase principal **Exception**

```
class SaldoInsuficienteError(Exception):  
    pass  
  
def retirar_dinero(saldo_actual, monto):  
    if monto > saldo_actual:  
        raise SaldoInsuficienteError(f"Fondos Insuficientes. Su saldo es {saldo_actual}")  
    return saldo_actual - monto  
  
try:  
    retirar_dinero(100, 150)  
except SaldoInsuficienteError as error:  
    print(f"Error detectado: {error}")
```

## Mejorando la excepción (sobrescribiendo `__str__` o `__init__`)

- Se puede agregar atributos adicionales (como un código de error específico) o sobrescribir el método `__str__` para tener un control total sobre el mensaje que se imprime en consola.

```
class ErrorDeValidación(Exception):  
    def __init__(self, mensaje, codigo):  
        self.codigo = codigo  
        super().__init__(mensaje)  
    def __str__(self):  
        return f"[Error {self.codigo}] {self.args[0]}"  
  
def ingresar_nombre():  
    nombre=input(f"Ingresa el nombre de usuario: ")  
    if len(nombre)<4:  
        raise ErrorDeValidación("El nombre de usuario es muy corto ", 401)  
try:  
    ingresar_nombre()  
except ErrorDeValidación as error:  
    print(f"Error detectado: {error}")
```

## Actividad 1

### Sistema de Cajero Automático

Desarrollar una aplicación que:

- Solicite monto de retiro.
- No permita valores negativos.
- No permita retirar más saldo disponible.
- Controle errores de entrada.

## ¿Qué es un depurador de código?

Es el proceso de identificar, analizar y corregir errores en el código fuente de un programa para asegurar su correcto funcionamiento.

Implica varios pasos importantes:

- **Reproducción del Error:** El primer paso es reproducir el error en un entorno controlado para observar en qué condiciones ocurre. Esto ayuda a entender el comportamiento del fallo.
- **Análisis del Código:** Una vez que se ha identificado el error, se analiza cómo responde el código ante ese fallo. Se utilizan herramientas como depuradores, logs y breakpoints para seguir la ejecución del código y verificar los valores de las variables
- **Implementación de Soluciones:** Después de analizar el comportamiento del código, se implementan soluciones para corregir los errores detectados. Esto puede incluir la modificación del código, la eliminación de líneas problemáticas o la reestructuración de funciones.

## ¿Cuándo es útil usar un depurador (Debugger)?

- **Aislar errores complejos:** Cuando tienes bloques de código largos o anidados y no sabes exactamente en qué línea ocurre el fallo.
- **Inspeccionar el valor de variables:** Esencial para ver cómo cambian los datos (como listas, diccionarios u objetos) línea por línea sin modificar tu código original.
- **Analizar la pila de llamadas (Call Stack):** Vital para saber qué función llamó a cuál y entender el historial de ejecución cuando recibes un error profundo.
- **Ejecución paso a paso:** Permite avanzar instrucción por instrucción para comprobar si el flujo lógico del programa sigue el camino correcto.
- **Modificar el estado en tiempo real:** Herramientas como PyCharm te permiten cambiar el valor de una variable mientras el programa está en pausa y ver cómo reacciona el código.



## Depuración (Debugging)

En Python se dispone:

- **PDB (Python Debugger):** Es el depurador nativo. Puedes pausar la ejecución insertando `import pdb; pdb.set_trace()` en tu código.
- **Depuración con Pytest:** Puedes combinarlo con `pytest --pdb`, lo cual le indica al framework que detenga la ejecución y abra una terminal interactiva en el momento exacto en que falla una prueba.

## Depuración (Debugging)

Uso de breakpoints:

- **Seleccionar líneas de código.** Presionar F9 para agregar un breakpoint
- Usar opciones:
  - Step over: F10
  - Step into: F11
  - Continue: F5

## Logging

- Muchos programadores necesitan registrar algunas de las acciones que sigue el código y que necesitamos que se registre/muestre.
- Es un sistema estándar integrado para registrar eventos, errores y datos de ejecución de un programa.
- Permite almacenar información útil en archivos o en la consola de manera profesional, superando las limitaciones de la función básica `print()`.
- Sus componentes clave son los niveles de gravedad, los manejadores (handlers) y los formateadores (formatters).
- Fomenta mejores prácticas.

| Nivel    | Uso                   |
|----------|-----------------------|
| DEBUG    | Información detallada |
| INFO     | Eventos normales      |
| WARNING  | Advertencias          |
| ERROR    | Errores               |
| CRITICAL | Fallos graves         |

## Tipos de salidas principales del Logging

- La salida de logging puede ser a varios destinos
- **Consola (Salida estándar):** Muestra los datos directamente en la pantalla usando **StreamHandler** o la configuración básica por defecto.
- **Archivo de texto:** Almacena el historial persistente de la aplicación usando **FileHandler** con modos como escritura (w) o adición (a).
- **Múltiples destinos:** Permite enviar los registros de manera simultánea tanto a la pantalla como a uno o varios archivos externos

## Logging

- Importar módulo

```
import logging
```

- Configuración Básica

```
logging.basicConfig(level=logging.INFO)

logging.info("Programa iniciado")
```

- Ejemplos:

```
logging.debug("Variable x = 10")

logging.info("Inicio correcto")

logging.warning("Espacio insuficiente")

logging.error("Error de conexión")
```

- Guardar logs en archivo

```
logging.basicConfig(
    filename="sistema.log",
    level=logging.INFO
)
```

## Ejemplo

- Configura un log para la app, también define el formato del log

```
import logging

logger = logging.getLogger("app")
logger.setLevel(logging.DEBUG)
handler = logging.FileHandler("app.log")
formatter = logging.Formatter('%(asctime)s | %(name)s | %(levelname)s | %(message)s')
handler.setFormatter(formatter)
logger.addHandler(handler)

def ejemplo():
    logger.info("= ejemplo =")
    print("hola")

if __name__ == "__main__":
    ejemplo()
```

- Salida

```
2026-07-26 14:14:42,666 | app | INFO | = ejemplo =
```

## Actividad 2

### Sistema de Login

- Registrar:
- Inicio de sesión exitoso.
- Usuario incorrecto.
- Contraseña incorrecta.
- Intentos fallidos.

### Guardar en:

- Login.log

## Testing con Pytest

Pytest es el estándar en el ecosistema de Python para realizar desde pruebas unitarias simples hasta complejas pruebas funcionales.

- **Estructura básica:** Los archivos de prueba deben nombrarse como `test_*.py` y las funciones de prueba deben comenzar con `test_`. Para verificar el resultado, se utilizan afirmaciones nativas de Python (`assert`).
- **Fixtures (`@pytest.fixture`):** Se utilizan para configurar y preparar el entorno antes de una prueba (como crear conexiones a bases de datos) y limpiar después, favoreciendo la reutilización.
- **Parametrización (`@pytest.mark.parametrize`):** Permite ejecutar una misma prueba múltiples veces con diferentes entradas y salidas esperadas, evitando la duplicación de código.
- **Excepciones esperadas (`with pytest.raises(...)`):** Verifica que tu código lance el error correcto ante situaciones inválidas o entradas defectuosas.



## Ejemplo con PyTest

### Operaciones.py

```
def sumar(a, b):
    return a + b
```

### Resultados:

### test\_operaciones.py

```
def test_Sumar():
    assert sumar(2, 3) == 5

def test_Sumar_negativos():
    assert sumar(-1, 1) == 0
```

```
===== FAILURES =====
test_Sumar

def test_Sumar():
> assert sumar(2, 3) == 5
E       assert 4 == 5
E       + where 4 = sumar(2, 3)

test_operaciones.py:8: AssertionError

test_Sumar_negativos

def test_Sumar_negativos():
> assert sumar(-1, 1) == 0
E       assert -1 == 0
E       + where -1 = sumar(-1, 1)

test_operaciones.py:11: AssertionError

===== short test summary info =====
FAILED test_operaciones.py::test_Sumar - assert 4 == 5
FAILED test_operaciones.py::test_Sumar_negativos - assert -1 == 0
2 failed in 0.40s
```

```
PS G:\Mi unidad\Curso Python\proyectos\poligono> pytest
===== test session starts =====
platform win32 -- Python 3.10.1, pytest-9.1.1, pluggy-1.6.0
rootdir: G:\Mi unidad\Curso Python\proyectos\poligono
collected 2 items

test_operaciones.py .. [100%]

===== 2 passed in 0.19s =====
PS G:\Mi unidad\Curso Python\proyectos\poligono>
```

## Ejemplo con PyTest

Para ejecutar los casos de prueba, en la terminal se indica a pytest que archivo se quiere ejecutar, o que carpeta contiene los casos de prueba

- Pytest # busca en el directorio actual todos los archivos que empiezan con test\_ y contienen pruebas para ejecutarlos
- Pytest ./pruebas # busca en la carpeta pruebas
- Pytest ./test\_operaciones.py # ejecuta las pruebas que existen en ese archivo
- Pytest ./test\_operaciones.py -collect-only #muestra los casos de prueba que existan dentro del archivo indicado
- Pytest ./test\_operaciones.py::test\_sumar\_negativos #ejecuta sólo ese caso de prueba

## Los 5 Principios SOLID

Los principios SOLID en Python son cinco reglas de diseño de código para lograr programas limpios, fáciles de mantener y escalar:

- **S - Responsabilidad Única (Single Responsibility Principle):** Una clase debe tener una sola razón para cambiar, es decir, debe hacer una sola tarea o tener un solo propósito.
- **O - Abierto/Cerrado (Open/Closed Principle):** El código debe estar abierto para crecer o agregar cosas nuevas, pero cerrado para cambiar lo que ya funciona.
- **L - Sustitución de Liskov (Liskov Substitution Principle):** Si usas una clase base, debes poder cambiarla por una clase hija sin que el programa falle o se comporte de forma extraña.
- **I - Segregación de Interfaces (Interface Segregation Principle):** Es mejor tener varias clases pequeñas y específicas que una sola clase gigante con métodos que no se usan.
- **D - Inversión de Dependencias (Dependency Inversion Principle):** Las partes importantes del sistema no deben depender de los detalles pequeños, sino de ideas abstractas o moldes generales.

## Tarea

El programa deberá mostrar un menú interactivo similar a:

===== HOTEL PARADISE INN =====

1. Mostrar habitaciones
2. Realizar reserva
3. Cancelar reserva
4. Consultar reservas
5. Salir

Seleccione una opción:

## Tarea

**Clase Reserva.** Debe contener:

- huésped
- habitación
- fecha de ingreso
- fecha de salida
- número de noches
- costo total

Debe incluir métodos como:

- confirmar\_reserva()
- cancelar\_reserva()
- mostrar\_reserva()

## Tarea

### Requerimientos:

1. **Clase abstracta.** Crear una clase abstracta llamada Habitación que represente una habitación genérica. Debe contener como mínimo los atributos: número, precio por noche, capacidad, disponible.

- Métodos abstractos: `calcular_precio(dias)` `mostrar_informacion()`

2. **Herencia.** Crear al menos tres clases que hereden de Habitación. Ejemplo: HabitaciónSimple, HabitaciónDoble y Suite

Cada tipo de habitación tendrá reglas diferentes para calcular el precio. Ejemplo:

- Habitación Simple → precio normal.
- Habitación Doble → descuento del 10% para reservas de más de 5 noches.
- Suite → incremento del 20% por servicios VIP.

3. **Polimorfismo.** El sistema debe utilizar polimorfismo al calcular el precio de las habitaciones para la opción 1. Cada objeto debe ejecutar su propia implementación del método

## Tarea

**Validaciones.** El sistema debe validar como mínimo:

- **Nombre del Húesped:** mínimo 3 caracteres, solo letras
- **Número de noches:** debe ser mayor que cero.
- **Disponibilidad:** no se puede reservar una habitación ocupada.

**Manejo de Excepciones.** Crear excepciones personalizadas. Como mínimo:

- `DatosInvalidosError`: Cuando algún dato ingresado no sea válido.
- `HabitacionNoDisponibleError`: Cuando se intente reservar una habitación ocupada.
- `FechaInvalidaError`: Cuando la fecha de salida sea anterior a la fecha de ingreso.
- `ReservaNoEncontradaError`: Cuando se intente cancelar una reserva inexistente.

Todas las excepciones deben ser capturadas utilizando bloques **try except** y mostrar mensajes amigables al usuario.

## Tarea

### Uso de Logging

Crear un archivo llamado hotel.log. Registrar al menos:

- inicio del programa
- creación de reservas
- cancelación de reservas
- errores de validación
- excepciones capturadas
- cierre del sistema

Utilizar diferentes niveles de logging:

- INFO
- WARNING
- ERROR



## Tarea

### Casos de Prueba

Se deberán diseñar y ejecutar casos de prueba que permitan verificar el correcto funcionamiento del sistema. Como mínimo se deberán documentar los siguientes tests:

| Caso                                          | Entrada                       | Resultado esperado                                      |
|-----------------------------------------------|-------------------------------|---------------------------------------------------------|
| Registrar un huésped con nombre de 2 letras   | "A1"                          | Se genera DatosInvalidosError                           |
| Reservar una habitación disponible            | Habitación libre              | Reserva creada correctamente                            |
| Reservar una habitación ocupada               | Habitación reservada ya       | Se genera HabitacionNoDisponibleError                   |
| Reservar con fechas inválidas                 | Salida anterior al ingreso    | Se genera FechaInvalidaError                            |
| Cancelar una reserva existente                | Código válido                 | Reserva cancelada correctamente                         |
| Cancelar una reserva inexistente              | Código inválido               | Se genera ReservaNoEncontradaError                      |
| Calcular el precio de cada tipo de habitación | Simple, Doble y Suite         | Cada una aplica su propia regla (polimorfismo)          |
| Revisar el archivo hotel.log                  | Después de varias operaciones | Se registran correctamente todas las acciones y errores |

## Tarea

**Fecha de Entrega:** Lunes: 3 d agosto

**Entregables:**

- Código fuente.
- Capturas de ejecución.
- Archivo de logs generado.
- Evidencia de pruebas exitosas con Pytest.

# GRACIAS